## Ejercicio 1:

Analice y compile el archivo ex_01.c.

1. Inspeccione el código y determine el objetivo del programa.
2. Analice los valores de las variables f1, f2, d1, acum_1 y acum_2.
3. ¿Qué conclusión puede obtener a partir de estos valores?

ex_01.c ● ex_02.c ex_04.c ex_05.c

home > garcia-14077 > Escritorio > Facultad > Quinto > CyS > TP3 > ex_01.c > ...

```c
// Compile usando el siguiente comando
// compile: gcc -Wall -std=c99 ex_01.c -o ex_01
#include <stdio.h>
#include <float.h>
#include <math.h>
#include <fenv.h>

typedef long int int64_t;

int main(void)
{
    float a, b, c, f1, f2;
    double d1;

    a = 1000000000.0;   // mil millones
    b =   20000000.0;   // 20 millones
    c =   20000000.0;
    f1 = (a * b) * c;
    f2 = a * (b * c);
    d1 = (double) (a) * (double) (b) * (double) (c);

    printf("f1 = %lf \n", f1 );
    printf("f2 = %lf \n", f2 );
    printf("d1 = %lf \n", d1 );

    printf("Error en f1 = %10e \n", f1 - 40000000000000000000000.0 );
    printf("Error en f2 = %10e \n", f2 - 40000000000000000000000.0 );
    printf("Error en d1 = %20e \n", d1 - 40000000000000000000000.0 );
    double acum_1, acum_2;

    acum_1 = 0.0;
    for (int64_t i = 0; i < 10000000; i++){ acum_1 += 0.01; }

    acum_2 = 0.0;
    b = 0.333;
    for (int64_t i = 0; i < 10000000; i++){ acum_2 += b / b; }

    printf("acum_1 = %f \n", acum_1 );
    printf("acum_2 = %f \n", acum_2 );
    printf("Error en acum_1 = %10e \n", acum_1 - (100000.0));
    printf("Error en acum_2 = %10e \n", acum_2 - (10000000.0));

    return 0;
}
```

```
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ gcc -Wall -std=c99 ex_01.c -o ex_01
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ /ex_01
bash: /ex_01: No existe el archivo o el directorio
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ ./ex_01
f1 = 4000000271566493534126 08.000000
f2 = 3999999911278523344486 40.000000
d1 = 3999999999999999966445568.000000
Error en f1 = 2.715665e+16
Error en f2 = -8.872148e+15
Error en d1 =         0.000000e+00
acum_1 = 99999.999986
acum_2 = 10000000.000000
Error en acum_1 = -1.369031e-05
Error en acum_2 = 0.000000e+00
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$
```

1) El objetivo principal de este código es demostrar las limitaciones del formato de representación en punto flotante. Específicamente, busca evidenciar dos grandes problemas de precisión:
    - La pérdida de la propiedad asociativa: Demuestra que en la aritmética de punto flotante, (a * b) * c no siempre es igual a a * (b * c).
    - El error de representación y su acumulación: Muestra cómo valores que son exactos en el sistema decimal (como 0.01) no tienen una representación finita exacta en binario, lo que provoca que los errores de redondeo se acumulen drásticamente al realizar iteraciones (como en un ciclo for).

2) Análisis de los valores de las variables:

- f1 y f2 (float): La variable float corresponde al formato "Single precision" de 32 bits, que tiene una precisión p = 24 bits para la mantisa (unos 7 dígitos decimales). El resultado exacto de la multiplicación es 4 * 10^{23}. Este número es tan grande que no entra entero en la mantisa, por lo que la computadora debe aplicar un esquema de redondeo (generalmente al más próximo) en cada paso intermedio.
    - En f1 = (a * b) * c, primero se redondea a * b y a ese resultado se lo multiplica por c (volviendo a redondear).
    - En f2 = a * (b * c), primero se redondea b * c y luego se multiplica por a. Al hacer los cortes de precisión en momentos distintos, f1 y f2 terminaron imprimiendo resultados diferentes.
- d1 (double): Utiliza el formato "Double precision" de 64 bits, con una precisión p = 53 bits. Al tener mucha más capacidad en el significando (mantisa), el error respecto al valor real de 4 *10^{23} va a ser significativamente menor, demostrando la diferencia de resolución. (Se puede observar una mejora de un orden de hasta 7 valores comparando con f1 y f2)
- acum_1: Suma 0.01 diez millones de veces. El problema es que 0.01 en binario es un número periódico. Como la representación es incompleta por ser finita, se guarda una aproximación. Ese pequeñísimo error inicial se suma 10 millones de veces, haciendo que el resultado final se aleje bastante de los 100000.0 teóricos.
- acum_2: Suma b / b diez millones de veces. Como 0.333 / 0.333 da exactamente 1.0, y el 1 sí tiene una representación exacta en binario normalizado (1.0* 2^0), en este caso no hay error de representación inicial. El resultado es exactamente 10000000.0 y el error es nulo.

3) Conclusiones a partir de los resultados

- Pérdida de la propiedad asociativa: Al observar f1 y f2, vemos que arrojan resultados y errores diferentes (2.715665e+16 frente a -8.872148e+15). Esto demuestra que en la aritmética de punto flotante, matemáticamente (a * b) * c es diferente a a * (b * c). Cómo estas variables utilizan el formato binario de 32 bits ("Single precision"), cuentan con una precisión en la mantisa de p = 24 bits. Al operar con números tan grandes, el resultado excede esta capacidad, obligando a la computadora a aplicar un esquema de redondeo en cada multiplicación intermedia. El orden en que se hacen esos redondeos altera el resultado final.
- Ventaja del formato de doble precisión: La variable d1 dio un error nulo (0.000000e+00)(Aunque la diferencia marca nulo, al observar el número podemos ver la existencia de un error, esto se debe a la incapacidad de la computadora de representar el número en cuestion). Al utilizar el formato "Double precision" de 64 bits, cuenta con una precisión mucho mayor de p = 53 bits. Esta mayor cantidad de bits le permitió absorber las multiplicaciones de este ejercicio en particular sin sufrir pérdida de información.
- Errores por representación incompleta: El valor de acum_1 resultó en 99999.999986 en lugar de 100000 exactos, arrastrando un error de -1.369031e-05. Esto ilustra que la representación en punto flotante es incompleta. El valor decimal 0.01 no tiene una representación binaria exacta, por lo que el sistema guarda la aproximación más cercana. Al sumar ese error minúsculo 10 millones de veces dentro del ciclo for, el error acumulado se vuelve significativo.
- Representación exacta de ciertos valores: Por el contrario, acum_2 arrojó un resultado perfecto. Esto ocurre porque la operación b / b da como resultado exacto 1. El número 1 sí posee una representación normalizada exacta, donde el bit oculto de la mantisa es implícitamente igual a 1 y se ajusta la parte fraccionaria a 0. Sumar un valor que carece de error de representación inicial no genera desvíos, por más que se itere millones de veces.

## Ejercicio 2:

Analice y compile el archivo ex_02.c.

1. ¿Para qué sirven las funciones fegetround() y fesetround()?
2. ¿Cuál es el modo de redondeo por defecto con el que arranca el programa?
3. Ejecutar el programa para los modos de redondeo FE_DOWNWARD, FE_UPWARD y FE_TOWARDZERO, y compárelos con el modo FE_TONEAREST.
4. ¿Observa diferencias? ¿Estas diferencias son consistentes con los modos de redondeos?

C ex_01.c  C ex_02.c ●  C ex_04.c  C ex_05.c

home > garcia-14077 > Escritorio > Facultad > Quinto > CyS > TP3 > C ex_02.c > ...

```c
// Compile usando el siguiente comando
// compile: gcc -Wall -std=c99 ex_02.c -o ex_02 -lm -frounding-math -fsignaling-nans

#include <stdio.h>
#include <float.h>
#include <math.h>
#include <fenv.h>

void test_rounding (void)
{
  float fp1, fp2;
  double df3;

 fp1 = exp(1.1);
 printf(" exp(1.1) = %+0.8f\n", fp1);

 fp2 = -exp(1.1);

  printf("-exp(1.1) = %+0.8f\n", fp2);

 df3 = exp(1.1);

  printf("double exp(1.1) = %+0.8lf\n", df3);
}

void current_rounding (void)
{
    int ROUND_MODE;

    ROUND_MODE = fegetround();

    printf("Current Round Mode = %d \n", ROUND_MODE );
}

void change_rounding (int rounding_mode)
{
    fesetround (rounding_mode);
}

int main(void)
{
    printf("** Floating-point single-precision constants ** \n");
    printf("FLT_MIN     = %E \n",       FLT_MIN );
    printf("FLT_MAX     = %E \n",       FLT_MAX );
    printf("FLT_EPSILON = %E \n",   FLT_EPSILON ); // En MATLAB: eps(single(1))
    printf("\n");

    printf("** Floating-point double-precision constants ** \n");
    printf("DBL_MIN     = %E \n",       DBL_MIN );
    printf("DBL_MAX     = %E \n",       DBL_MAX );
    printf("DBL_EPSILON = %E \n",   DBL_EPSILON ); // En MATLAB: eps(1)
    printf("\n");

    printf("** Floating point rounding modes ** \n");
    printf("Rounding Mode FE_TONEAREST  = %d \n",   FE_TONEAREST );
    printf("Rounding Mode FE_DOWNWARD   = %d \n",       FE_DOWNWARD );
    printf("Rounding Mode FE_UPWARD     = %d \n",       FE_UPWARD );
    printf("Rounding Mode FE_TOWARDZERO = %d \n",   FE_TOWARDZERO );
    printf("\n");
```

```c

    current_rounding();
    test_rounding();
    printf("\n");

    change_rounding(FE_DOWNWARD);
    current_rounding();
    test_rounding();
    printf("\n");

    change_rounding(FE_UPWARD);
    current_rounding();
    test_rounding();
    printf("\n");

    change_rounding(FE_TOWARDZERO);
    current_rounding();
    test_rounding();
    printf("\n");
    return 0;
}
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ gcc -Wall -std=c99 ex_02.c -o ex_02 -lm -frounding-math -fsignaling-nans
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ ./ex_02
** Floating-point single-precision constants **
FLT_MIN         = 1.175494E-38
FLT_MAX         = 3.402823E+38
FLT_EPSILON     = 1.192093E-07

** Floating-point double-precision constants **
DBL_MIN         = 2.225074E-308
DBL_MAX         = 1.797693E+308
DBL_EPSILON     = 2.220446E-16

** Floating point rounding modes **
Rounding Mode FE_TONEAREST      = 0
Rounding Mode FE_DOWNWARD       = 1024
Rounding Mode FE_UPWARD         = 2048
Rounding Mode FE_TOWARDZERO     = 3072

Current Round Mode = 0
 exp(1.1) = +3.00416613
-exp(1.1) = -3.00416613
double exp(1.1) = +3.00416602

Current Round Mode = 1024
 exp(1.1) = +3.00416588
-exp(1.1) = -3.00416613
double exp(1.1) = +3.00416602

Current Round Mode = 2048
 exp(1.1) = +3.00416613
-exp(1.1) = -3.00416588
double exp(1.1) = +3.00416603

Current Round Mode = 3072
 exp(1.1) = +3.00416588
-exp(1.1) = -3.00416588
double exp(1.1) = +3.00416602

garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$
```

1) La función fegetround() se utiliza para obtener el modo actual configurado en el entorno para operaciones de punto flotante. Por su parte, fesetround() permite establecer un nuevo esquema de redondeo, el cual define el criterio y procedimiento para redondear el resultado x de una operación en punto flotante, ya sea al valor f ’ o f ".

2) El programa inicia por defecto con el modo FE_TONEAREST (representado por el valor entero 0 en la terminal). Este modo corresponde al "redondeo al más próximo".

3) A partir de la terminal, los resultados para la variable de precisión simple (float) fp1 = exp(1.1) y fp2 = -exp(1.1) son:

- FE_TONEAREST (0): fp1 = +3.00416613 | fp2 = -3.00416613
- FE_DOWNWARD (1024): fp1 = +3.00416588 | fp2 = -3.00416613
- FE_UPWARD (2048): fp1 = +3.00416613 | fp2 = -3.00416588
- FE_TOWARDZERO (3072): fp1 = +3.00416588 | fp2 = -3.00416588

4) Sí, se observan diferencias en los últimos dígitos de la representación de precisión simple y son completamente consistentes con la teoría matemática de los esquemas de redondeo:

- Redondeo hacia menos infinito (FE_DOWNWARD): Aproxima al valor f ‘. Por esto, el valor positivo fp1 se reduce al límite inferior (+3.00416588), mientras que el negativo fp2 adopta el valor más negativo posible (-3.00416613).
- Redondeo hacia más infinito (FE_UPWARD): Aproxima al valor f “. Aquí ocurre lo inverso: fp1 adopta el valor superior (+3.00416613) y fp2 toma el valor menos negativo (-3.00416588).
- Truncado / Redondeo hacia cero (FE_TOWARDZERO): El resultado se aproxima a f’ si x > 0, y a f ’’ si x < 0. Esto se verifica en la salida, ya que tanto el valor positivo como el negativo adoptan la representación de menor magnitud absoluta .

## Ejercicio 3:

Investigue cómo inicializar en C variables en punto flotante con los valores especiales NaN, Inf y -Inf. Además, escriba un programa en C que produzca como resultado estos valores especiales.

1. Representación teórica (Estándar IEEE 754-2008):

Para fundamentar el comportamiento de las variables, es necesario referirse a la estructura de bits del formato, compuesta por el signo S, el exponente sesgado E y la mantisa T. Los valores especiales se codifican saturando el campo del exponente:

- Infinito (Inf): Se representa con todos los bits del exponente en 1 (E=111...1) y la fracción o mantisa en 0 (T=0). El bit de signo S determina si se trata de +Inf (S=0) o -Inf (S=1).
- Not a Number (NaN): Se representa con todos los bits del exponente en 1 (E=111...1), pero con la fracción distinta de cero (T distinto de 0). El estándar utiliza este estado para representar resultados de operaciones que no están definidas en el dominio de los números reales.

2. Inicialización explícita en lenguaje C:

Para inicializar variables con estos valores de forma manual, el estándar C99 proporciona macros específicas definidas en la cabecera `<math.h>`:

- `INFINITY`: Expande a una expresión constante de tipo `float` que representa el infinito positivo. Para el infinito negativo, simplemente se aplica el operador unitario (`-INFINITY`).

- NAN: Expande a una expresión constante que representa un valor "Not a Number".

3. Generación mediante operaciones matemáticas

A nivel de hardware, las ALU (Unidades Aritmético Lógicas) o coprocesadores matemáticos generan estos valores automáticamente al procesar indeterminaciones numéricas. Esto es análogo al comportamiento que se observa en entornos de cálculo matricial orientados a ingeniería.

ex_01.c | ex_02.c | ex_03.c | ex_04.c | ex_05.c

home > garcia-14077 > Escritorio > Facultad > Quinto > CyS > TP3 > ex_03.c > main(void)

```c
// Compilar con: gcc -Wall -std=c99 ex_03.c -o ex_03 -lm
#include <stdio.h>
#include <math.h>

int main(void) {
    // 1. Inicialización directa (Macros de math.h)
    double var_inf_pos = INFINITY;
    double var_inf_neg = -INFINITY;
    double var_nan = NAN;

    printf("--- Inicializacion Directa ---\n");
    printf("Infinito positivo: %f\n", var_inf_pos);
    printf("Infinito negativo: %f\n", var_inf_neg);
    printf("Not a Number (NaN): %f\n\n", var_nan);

    // 2. Generación por cálculo en tiempo de ejecución
    // Se utilizan variables en lugar de literales (ej. 1.0/0.0)
    // para forzar el cálculo en la ALU y evitar warnings del compilador.
    double zero = 0.0;
    double num = 1.0;
    double neg_num = -1.0;

    printf("--- Generacion por Calculo ---\n");

    // +Inf: División de un real positivo por cero
    double calc_inf = num / zero;
    printf("1.0 / 0.0 = %f\n", calc_inf);

    // -Inf: División de un negativo por cero, o logaritmo natural de cero
    double calc_ninf_1 = neg_num / zero;
    double calc_ninf_2 = log(zero);
    printf("-1.0 / 0.0 = %f\n", calc_ninf_1);
    printf("log(0.0)  = %f\n", calc_ninf_2);

    // NaN: Indeterminaciones matemáticas (0/0 o Inf/Inf)
    double calc_nan_1 = zero / zero;
    double calc_nan_2 = calc_inf / calc_inf;
    printf("0.0 / 0.0 = %f\n", calc_nan_1);
    printf("Inf / Inf = %f\n", calc_nan_2);

    return 0;
}
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ gcc -Wall -std=c99 ex_03.c -o ex_03 -lm
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ ./ex_03
--- Inicializacion Directa ---
Infinito positivo: inf
Infinito negativo: -inf
Not a Number (NaN): nan

--- Generacion por Calculo ---
1.0 / 0.0 = inf
-1.0 / 0.0 = -inf
log(0.0)  = -inf
0.0 / 0.0 = -nan
Inf / Inf = -nan
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$
```

4. Paralelismo con entornos de cálculo matricial

El comportamiento a nivel de hardware observado en C es el mismo que subyace en entornos de ingeniería orientados a matrices, como MATLAB. El estándar IEEE-754 garantiza que, independientemente del lenguaje, el hardware capture las operaciones indefinidas. Por ejemplo, al igual que en C, si en un entorno de alto nivel se evalúa una función que excede el rango dinámico máximo representable (como exp(1000)), el sistema genera un *overflow*, saturando los bits del exponente para devolver Inf . De la misma manera, las indeterminaciones como 0/0 o Inf/Inf son procesadas uniformemente devolviendo la constante NaN.

## Ejercicio 4:

Analice y compile el archivo ex_04.c.

1. Analice los resultados impresos por consola ¿Son los resultados consistentes con las operaciones ejecutadas?
2. Explique qué hacen las funciones feclearexcept(), feraiseexcept() y fetestexcept().

```
ex_01.c    ex_02.c    ex_03.c    ex_04.c ×    ex_05.c
home > garcia-14077 > Escritorio > Facultad > Quinto > CyS > TP3 > ex_04.c > ...
#include <stdio.h>
#include <float.h>
#include <math.h>
#include <signal.h>
#include <stdlib.h>

// Compile usando el siguiente comando
// compile: gcc -Wall -O3 -std=c99 ex_04.c -o ex_04 -lm -march=corei7 -frounding-math -fsignaling-nans

#define _GNU_SOURCE 1
#define _ISOC99_SOURCE
#include <fenv.h>

void show_fe_exceptions(void)
{
    printf("current exceptions raised: ");
    if(fetestexcept(FE_DIVBYZERO))     printf(" FE_DIVBYZERO");
    if(fetestexcept(FE_INEXACT))       printf(" FE_INEXACT");
    if(fetestexcept(FE_INVALID))       printf(" FE_INVALID");
    if(fetestexcept(FE_OVERFLOW))      printf(" FE_OVERFLOW");
    if(fetestexcept(FE_UNDERFLOW))     printf(" FE_UNDERFLOW");
    if(fetestexcept(FE_ALL_EXCEPT)==0) printf(" none");
    printf("\n");
}

```

```c
int main(void)
{
  int ROUND_MODE;

  ROUND_MODE = fegetround();
  printf("Current Round Mode = %d \n", ROUND_MODE );

  show_fe_exceptions();

  /* Temporarily raise other exceptions */
  feclearexcept(FE_ALL_EXCEPT);
  feraiseexcept(FE_INEXACT);
  show_fe_exceptions();

  feclearexcept(FE_ALL_EXCEPT);
  feraiseexcept(FE_INVALID);
  show_fe_exceptions();

```

```c

  feclearexcept(FE_ALL_EXCEPT);
  feraiseexcept(FE_DIVBYZERO);
  show_fe_exceptions();

  feclearexcept(FE_ALL_EXCEPT);
  feraiseexcept(FE_OVERFLOW);
  show_fe_exceptions();

  feclearexcept(FE_ALL_EXCEPT);
  feraiseexcept(FE_UNDERFLOW);
  show_fe_exceptions();

  feclearexcept(FE_ALL_EXCEPT);
  feraiseexcept(FE_OVERFLOW | FE_INEXACT);
  show_fe_exceptions();

  return 0;
}
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                                    bash - TF

garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ gcc -Wall -O3 -std=c99 ex_04.c -o ex_04 -lm -march=corei7 -frounding-math -fsignaling-nans
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ ./ex_04
Current Round Mode = 0
current exceptions raised:  none
current exceptions raised:  FE_INEXACT
current exceptions raised:  FE_INVALID
current exceptions raised:  FE_DIVBYZERO
current exceptions raised:  FE_OVERFLOW
current exceptions raised:  FE_UNDERFLOW
current exceptions raised:  FE_INEXACT FE_OVERFLOW
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$
```

1) Consistencia de los resultados:

Los resultados impresos por consola son completamente consistentes con las instrucciones ejecutadas. En este programa no se realizan operaciones aritméticas reales para inducir errores numéricos, sino que se manipulan de forma directa las banderas (*flags*) del entorno de punto flotante. La salida de la terminal refleja con exactitud la secuencia temporal de las activaciones manuales programadas en el código fuente.

2) Funciones de manejo de excepciones

- feclearexcept(): Limpia (restablece a cero) las banderas de las excepciones de punto flotante especificadas en su argumento. En el código, se utiliza con la macro FE_ALL_EXCEPT para limpiar todo el registro de excepciones antes de realizar la siguiente prueba.
- feraiseexcept(): Levanta (activa) manualmente las excepciones de punto flotante indicadas en su argumento. Esto instruye al entorno a simular que ha ocurrido dicho evento aritmético sin necesidad de ejecutar un cálculo fallido.

- fetestexcept(): Evalúa el estado actual del entorno y devuelve un valor distinto de cero si la bandera de la excepción especificada en su argumento se encuentra actualmente activada.

## Ejercicio 5:

Con el código visto en el ejercicio 4, ex_04.c. genere los 4 tipo de excepciones estudiadas al ejecutar diferentes operaciones matemáticas, en lugar de utilizar la función feraiseexcept()

ex_01.c | ex_02.c | ex_03.c | ex_04.c | ex_04b.c × | ex_05.c

home > garcia-14077 > Escritorio > Facultad > Quinto > CyS > TP3 > ex_04b.c > ...

```c
#include <stdio.h>
#include <float.h>
#include <math.h>
#include <fenv.h>

void show_fe_exceptions(void) {
    printf("current exceptions raised: ");
    if(fetestexcept(FE_DIVBYZERO))     printf(" FE_DIVBYZERO");
    if(fetestexcept(FE_INEXACT))       printf(" FE_INEXACT");
    if(fetestexcept(FE_INVALID))       printf(" FE_INVALID");
    if(fetestexcept(FE_OVERFLOW))      printf(" FE_OVERFLOW");
    if(fetestexcept(FE_UNDERFLOW))     printf(" FE_UNDERFLOW");
    if(fetestexcept(FE_ALL_EXCEPT)==0) printf(" none");
    printf("\n");
}

int main(void) {
    volatile double a, b, result;

    feclearexcept(FE_ALL_EXCEPT);
    printf("Generando FE_INVALID (0.0 / 0.0):\n");
    a = 0.0; b = 0.0;
    result = a / b;
    printf("Resultado = %f\n", result);
    show_fe_exceptions();

    feclearexcept(FE_ALL_EXCEPT);
    printf("\nGenerando FE_DIVBYZERO (1.0 / 0.0):\n");
    a = 1.0; b = 0.0;
    result = a / b;
    printf("Resultado = %f\n", result);
    show_fe_exceptions();

    feclearexcept(FE_ALL_EXCEPT);
    printf("\nGenerando FE_OVERFLOW (DBL_MAX * 2.0):\n");
    a = DBL_MAX; b = 2.0;
    result = a * b;
    printf("Resultado = %e\n", result);
    show_fe_exceptions();

    feclearexcept(FE_ALL_EXCEPT);
    printf("\nGenerando FE_UNDERFLOW (DBL_MIN / 1e10):\n");
    a = DBL_MIN; b = 1e10;
    result = a / b;
    printf("Resultado = %e\n", result);
    show_fe_exceptions();

    feclearexcept(FE_ALL_EXCEPT);
    printf("\nGenerando FE_INEXACT (1.0 / 3.0):\n");
    a = 1.0; b = 3.0;
    result = a / b;
    printf("Resultado = %f\n", result);
    show_fe_exceptions();

    return 0;
}
```

```
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ gcc -Wall -O3 -std=c99 ex_04b.c -o ex_04b -lm -march=corei7 -frounding-math -fsignaling-nans
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ ./ex_04b
Generando FE_INVALID (0.0 / 0.0):
Resultado = -nan
current exceptions raised:  FE_INVALID

Generando FE_DIVBYZERO (1.0 / 0.0):
Resultado = inf
current exceptions raised:  FE_DIVBYZERO

Generando FE_OVERFLOW (DBL_MAX * 2.0):
Resultado = inf
current exceptions raised:  FE_INEXACT FE_OVERFLOW

Generando FE_UNDERFLOW (DBL_MIN / 1e10):
Resultado = 2.225074e-318
current exceptions raised:  FE_INEXACT FE_UNDERFLOW

Generando FE_INEXACT (1.0 / 3.0):
Resultado = 0.333333
current exceptions raised:  FE_INEXACT
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$
```

A partir de la ejecución de operaciones aritméticas críticas, se verificó que la Unidad Aritmético Lógica (ALU) detecta y señaliza automáticamente las anomalías matemáticas de acuerdo con las especificaciones del estándar IEEE 754-2008 . El hardware activa las banderas de excepción correspondientes para advertir sobre las limitaciones inherentes a la representación finita , de la siguiente manera:

- Indeterminaciones (FE_INVALID): Al ejecutar operaciones no definidas en el dominio de los números reales (como $0.0 / 0.0$), el sistema asigna la codificación especial *Not a Number* (NaN) .
- Singularidades (FE_DIVBYZERO): La división de un número finito no nulo por cero fuerza al sistema a codificar el valor como Infinito (Inf), saturando todos los bits del campo del exponente .
- Desbordamiento superior (FE_OVERFLOW): Ocurre cuando el resultado de una operación excede el límite máximo del rango dinámico del formato . Al no disponer de suficientes bits para el exponente, la magnitud real se pierde.
- Desbordamiento inferior (FE_UNDERFLOW): Se produce al generar resultados de una magnitud tan pequeña que no pueden ser representados de forma normalizada. El sistema recurre a la representación denormalizada ($E=0$) , asumiendo una importante pérdida de precisión relativa .
- Inexactitud (FE_INEXACT): Evidencia que la representación en punto flotante es incompleta . Se activa invariablemente cuando el resultado analítico requiere más bits de mantisa de los disponibles, obligando al procesador a aplicar un esquema de redondeo (como la aproximación al valor representable más cercano) .

## Ejercicio 6:

Analice y compile el archivo ex_05.c.

1. Analice los resultados impresos por consola ¿Son los resultados consistentes con las operaciones ejecutadas?
2. Descomentar las líneas 44 y 45 y volver a compilar. ¿Qué observa por consola? ¿Cuál es la función de feenableexcept()?
3. Descomentar la línea 47 y volver a compilar.
4. ¿Qué observa por consola? ¿Cuál es el objetivo de la función signal(SIGFPE, fpe_handler)?

ex_01.c | ex_02.c | ex_03.c | ex_04.c | ex_04b.c | ex_05.c

home > garcia-14077 > Escritorio > Facultad > Quinto > CyS > TP3 > ex_05.c > ...

```c
1  // Version: 004
2  // Date:    2022/04/05
3  // Author:  Rodrigo Gonzalez <rodralez@frm.utn.edu.ar>
4
5  #define _GNU_SOURCE 1
6  #define _ISOC99_SOURCE
7  #include <fenv.h>
8
9  #include <stdio.h>
10 #include <float.h>
11 #include <math.h>
12 #include <signal.h>
13 #include <stdlib.h>
14
15 // Compile usando el siguiente comando
16 // compile: gcc -Wall -std=c99 ex_05.c -o ex_05 -lm -frounding-math -fsignaling-nans -DLINUX
17
18 // Variables globales
19 fexcept_t excepts;
20
```

```c
21  void show_fe_exceptions(void)
22  {
23      printf("current exceptions raised: ");
24      if(fetestexcept(FE_DIVBYZERO))     printf(" FE_DIVBYZERO");
25      if(fetestexcept(FE_INEXACT))       printf(" FE_INEXACT");
26      if(fetestexcept(FE_INVALID))       printf(" FE_INVALID");
27      if(fetestexcept(FE_OVERFLOW))      printf(" FE_OVERFLOW");
28      if(fetestexcept(FE_UNDERFLOW))     printf(" FE_UNDERFLOW");
29      if(fetestexcept(FE_ALL_EXCEPT)==0) printf(" none");
30      printf("\n");
31  }
32
33  void fpe_handler(int sig){
34
35    printf ("UPS! Floating Point Exception \n");
36
37    show_fe_exceptions();
38
39    exit(EXIT_FAILURE);
40  }
41
```

```c
42  int main(void)
43  {
44    //~ feclearexcept (FE_ALL_EXCEPT);
45    //~ feenableexcept(FE_ALL_EXCEPT);
46
47    //~ signal(SIGFPE, fpe_handler);
48
49    /* Setup a "current" set of exception flags. */
50    feraiseexcept(FE_INVALID);
51    show_fe_exceptions();
52
53    //~ /* Temporarily raise two other exceptions. */
54    feclearexcept(FE_ALL_EXCEPT);
55    feraiseexcept(FE_OVERFLOW | FE_INEXACT);
56    show_fe_exceptions();
57
58    //~ double s;
59    //  s = 1.0 / 0.0;  // FE_DIVBYZERO
60    //  s = 0.0 / 0.0;  // FE_INVALID
61    //  show_fe_exceptions();
62
63    return 0;
64  }
```

1) Analice los resultados impresos por consola ¿Son los resultados consistentes con las operaciones ejecutadas?

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ gcc -Wall -std=c99 ex_05.c -o ex_051 -lm -frounding-math -fsignaling-nans -DLINUX
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ ./ex_051
current exceptions raised:  FE_INVALID
current exceptions raised:  FE_INEXACT FE_OVERFLOW
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$
```

Los resultados impresos por consola en esta primera ejecución son totalmente consistentes con las operaciones codificadas. En esta instancia inicial, el programa no evalúa cálculos aritméticos reales, sino que manipula y evalúa las banderas de estado de manera manual:

- En primer lugar, la instrucción feraiseexcept(FE_INVALID) levanta explícitamente la bandera de la excepción de operación inválida. Al llamar a la función de inspección, la consola imprime correctamente FE_INVALID.
- A continuación, el programa limpia el registro de excepciones utilizando feclearexcept(FE_ALL_EXCEPT) y levanta simultáneamente las banderas correspondientes a desbordamiento superior e inexactitud con la instrucción feraiseexcept(FE_OVERFLOW | FE_INEXACT). La consola refleja exactamente este estado imprimiendo FE_INEXACT FE_OVERFLOW.

2) Descomentar las líneas 44 y 45 y volver a compilar. ¿Qué observa por consola? ¿Cuál es la función de feenableexcept()?

```
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ gcc -Wall -std=c99 ex_05.c -o ex_052 -lm -frounding-math -fsignaling-nans -DLINUX
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ ./ex_052
Excepción de coma flotante (`core' generado)
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$
```

Al recompilar y ejecutar el programa con las líneas 44 y 45 activas, se observa que la ejecución se interrumpe abruptamente y la consola devuelve el error Excepción de coma flotante ('core' generado). Ninguno de los mensajes anteriores programados con printf llega a mostrarse, ya que el programa es abortado.

La función feenableexcept() tiene como objetivo desenmascarar (habilitar) el lanzamiento de *traps* (interrupciones de hardware) ante excepciones de punto flotante. Por defecto, el estándar IEEE-754 establece que las excepciones matemáticas operan de forma silenciosa: simplemente levantan una bandera de estado interna y la ejecución del programa continúa con valores especiales como NaN o Infinito .

Al ejecutar feenableexcept(FE_ALL_EXCEPT), se instruye al procesador para que no actúe de forma silenciosa. En su lugar, en el instante exacto en que se levanta la bandera de operación inválida mediante feraiseexcept(FE_INVALID), el procesador detiene la ejecución y envía una señal de error fatal al sistema operativo (conocida en POSIX como SIGFPE). Como el programa aún no cuenta con un manejador (*handler*) definido para atrapar esta señal, el sistema operativo aplica su comportamiento por defecto: finalizar inmediatamente el proceso y volcar la memoria (*core dump*).

3) Descomentar la línea 47 y volver a compilar. ¿Qué observa por consola? ¿Cuál es el objetivo de la función signal(SIGFPE, fpe_handler)?

```
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ gcc -Wall -std=c99 ex_05.c -o ex_053 -lm -frounding-math -fsignaling-nans -DLINUX
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$ ./ex_053
UPS! Floating Point Exception
current exceptions raised:  none
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP3$
```

Al ejecutar el programa con la línea 47 activa, se observa por consola la impresión del mensaje personalizado "UPS! Floating Point Exception" seguido de "current exceptions raised: none". A diferencia del caso anterior, el programa finaliza de forma controlada sin generar un volcado de memoria (*core dump*).

El objetivo de la función signal(SIGFPE, fpe_handler) es registrar un manejador de señales (*signal handler*) personalizado. Sirve para interceptar la señal SIGFPE (Floating-Point Exception), que el sistema operativo envía cuando el procesador detecta una anomalía aritmética previamente habilitada por feenableexcept(). En lugar de aplicar el comportamiento por defecto del sistema (abortar el proceso de forma abrupta), signal redirige el flujo de control hacia la función fpe_handler. Esto permite al desarrollador ejecutar una rutina de emergencia, registrar el error y abortar el programa de manera segura mediante exit(EXIT_FAILURE).

*(Nota sobre el resultado "none": Al saltar al manejador de señales, el contexto del procesador cambia y, en muchas implementaciones de Linux/glibc, las banderas de estado de punto flotante se limpian o no son directamente accesibles desde la función manejadora simple, por lo que fetestexcept no detecta la excepción original).*